

Midterm Project: Exploring AI Agent Tool Calling

Team project: two students per team

1 Project Overview

Install and use **one** AI agent platform, either **Hermes Agent** or **OpenClaw**. Learn its main functions through hands-on tests, with particular attention to how the agent selects, invokes, and receives results from tools. Produce a high-level architecture diagram of the tool-calling mechanism and support the diagram with evidence from an actual agent run. Explain observed tool behavior; software for automatic graph generation is not required.

Work in a team of exactly two students. Both students must participate in installation, testing, analysis, reporting, and presentation. Teams receive the same project score when their documented contributions are substantially equal. The instructor may assign different individual scores when evidence shows unequal contributions.

2 Learning Objectives

By the end of the project, you should be able to:

1. Install and configure Hermes Agent or OpenClaw with a working language model.
2. Demonstrate basic agent functions and at least one actual tool call.
3. Explain the path from a user question through model tool selection, tool execution, returned observation, and final response.
4. Identify actual tool calls in run evidence and distinguish them from the agent's description of what it intended to do.
5. Draw and defend a high-level architecture of the tool-calling mechanism.

3 Platform and Model Access

Install and run your selected agent on the **University of Baltimore course VM**. Follow the instructor's VM access and setup instructions. Choose one of these model-access options:

1. **Course-provided model:** connect the agent on the VM to the small model provided by the university. This option requires no student purchase. Check whether the model performs the tool calls needed for your tests, and document any limitations you observe.
2. **Student-provided hosted API (optional):** connect the agent on the VM to a supported model provider using an API key you obtain yourself. A hosted model may respond faster or perform tool-use tasks better than the small course model, but this is not guaranteed. You are responsible for any charges; set a spending limit and use small test tasks. This option receives no extra credit.

Buying API tokens is not required. Students using the course-provided model are evaluated by the same rubric. Do not include keys or tokens in the report, screenshots, slides, or GitHub repository. Use environment variables or a local configuration file excluded from Git, and remove

your key from the shared VM when finished. Record the platform version or commit, VM environment, model/provider and version, installation method, and tool settings so another student can reproduce your work.

4 Shared Starter Dataset and Questions

Use the synthetic files and **midterm questions M0–M2** in `STARTER.md`. Copy the data folder identified there to your controlled VM workspace and give the agent read access. Run M0, M1, and M2 at least once using their supplied wording; replace `DATA_DIR` with your actual path. M3 is a suggested missing-record limitation case. Save the original prompts and observed call evidence. The guide provides expected factual answers, but it does not prescribe a tool-call sequence: report the calls that actually occurred. Document any changed prompt and its reason, especially if the small course model does not invoke a tool on the first attempt.

5 Required Work

5.1 Install and verify

Follow the official installation/onboarding instructions for Hermes or OpenClaw. Show that the agent starts, connects to the selected model, and answers a simple question. Document the steps you actually performed, relevant commands or UI settings, and any problems you solved. Redact private paths, addresses, and credentials from shared evidence.

5.2 Explore functions and test tools

Explore at least **three** functions or capabilities of your chosen agent. These may include interactive chat, tool listing or policy configuration, file access, command execution in a controlled workspace, web retrieval, session history, or built-in activity/trace views. At least **one** test must show an actual tool invocation in a run record or other direct evidence. The same tool may be called more than once; different tool types are not required. Use only course-approved or your own test data and a disposable workspace for commands or file operations.

Run at least **three distinct prompts or tasks**: (a) a basic model-response check, (b) a task that causes at least one tool call, and (c) a cross-file task in which you explain the tool calls that actually occurred. Multiple calls are useful to observe but are not required for the midterm. If the agent declines or fails to call a tool, record that result, explain why it may have happened, adjust the prompt or tool configuration, and run a further test. For each task, save the exact prompt, relevant settings, agent response, and evidence of what the agent actually did. Include at least one controlled case that shows a tool error, denied action, or other limitation if it can be produced safely.

5.3 Explain tool calling

Identify the main components of the mechanism for your installed version: user interface or gateway, conversation/context, model request and tool definitions, model-selected call, policy or permission check (if present), tool executor, returned result, and final model response. Explain which component chooses a tool, which component executes it, and how the result returns to the model. Describe what you can observe directly and what remains an inference.

Draw a **high-level architecture diagram** with labeled components and arrows. Include the agent on the course VM, the chosen model (course-provided or hosted), at least one enabled tool, and the path for a returned tool result. Add a numbered call sequence for one real test question. Mark where request, execution, result, and call identity can be observed. The architecture may be drawn manually or with a diagram tool, but its labels must be readable in the PDF and slides. This is a conceptual architecture informed by documentation and observed runs; do not present unverified internal details as facts.

Illustrative M1 sequence. Figure 1 shows how a question might lead to one file-tool call and a final answer. The model selects a tool, the agent runtime executes it, and the result returns to the model. Use evidence from your own run to describe what actually happened. This sequence is an example, not the high-level architecture diagram you must submit.

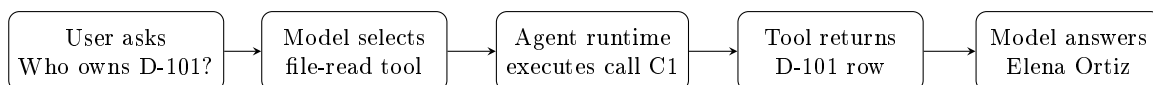


Figure 1: One possible M1 tool-calling sequence. The actual tool name, call ID, and evidence must come from the team’s run.

Hermes running example. Figure 2 shows part of a Hermes terminal session during a separate task. The highlighted lines show a process poll and wait, then a terminal command followed by another wait. This illustrates that one task may involve several tool invocations. The screenshot is only a partial view: it does not establish every call’s arguments, result, or complete order. For your own test, use the available run record to identify and explain the calls you actually observed.

```

Let me check on the install:

⚡ preparing tool_call...
⚙️ proc    poll proc_28a65b4  0.0s
⚡ preparing tool_call...
⚙️ proc    wait proc_28a65b4  6.8s

$ Hermes
Chatterbox installed. Converting the reference to 16kHz mono and running the synthesis (n):

🖥️ preparing terminal...
$ ffmpeg -y -i separated/htdemucs/wang_zhian_voice_sample/vocals.wav -ar
1 command 0.2s
⚡ preparing tool_call...
⚙️ proc    wait proc_e8804b4  17.2s

$ Hermes
Import name differs in this version – checking the API:

```

Figure 2: Example Hermes session showing visible tool-call activity. The highlighted status lines and terminal command are from an illustrative task, not the required M1 dataset question.

5.4 Assess observation limits

Identify one way to inspect tool-call evidence in your selected platform (for example, Hermes hooks or OpenClaw trajectory/audit records). State what it records, one visibility gap, and one question you could not resolve from the available evidence. Implementing a separate collector is not required.

6 Midterm Technical Report

Prepare a clear, professional technical report in a standard single-column format. Use an 11- or 12-point readable font, 1-inch margins, numbered section headings, page numbers, figure and table captions, and consistent citations and references. The report may be created in Word, Google Docs, Overleaf, or another suitable editor, but it must be submitted as a PDF. The main report must be **at least 6 pages**, excluding references and an optional appendix. There is no strict maximum, but the report should remain focused and avoid unnecessary repetition. Include representative screenshots and other essential run evidence in the main report; place additional supporting evidence in the optional appendix. Include:

1. **Title and team members:** project title, student names, selected platform, and model access path.
2. **Executive Summary:** tests performed, main findings, and the most important limitation observed.
3. **Introduction and Objectives:** purpose, scope, and team objectives.
4. **Platform and Setup:** agent version, system/VM, model/provider, installation and verification steps.
5. **Tool-Calling Architecture:** labeled high-level diagram, component roles, and numbered sequence for one real question.
6. **Test Methodology:** exact prompts, enabled tools, expected observations, and evidence source.
7. **Results and Evidence:** per-task table of actual behavior, tool names and outcomes, representative evidence, and discrepancies.
8. **Discussion and Limitations:** visibility gaps, unresolved questions, and how evidence could be improved.
9. **Conclusion:** what the team learned and what remains unresolved.
10. **References:** consistent citations to platform and model documentation and other sources.
11. **Appendix (optional):** additional redacted screenshots, command output, and detailed setup notes.

Include the team's **GitHub repository URL** and evaluated commit hash in the report. Include a short **team contributions and AI-assistance statement** identifying each student's work, any ChatGPT/Claude or other AI help, and how outputs were checked. AI assistance is encouraged, but the team must verify and understand all submitted material.

7 Deliverables and Submission

Each team must create a **GitHub repository** containing setup/reproduction instructions, a copy of the shared synthetic data, safe configuration examples, prompts, redacted run evidence, the architecture diagram source or image, and any test scripts. Make the repository accessible to the instructor by the deadline. Do not commit credentials, private data, or unrestricted logs that may contain them.

Each student must independently submit **one Canvas submission** containing:

1. The same team midterm technical report PDF.
2. A share link to the editable report source (for example, Word, Google Docs, or Overleaf) with instructor access.
3. The team’s presentation slides (PPT or PPTX).
4. The team’s GitHub repository link and evaluated commit hash.
5. A short individual contribution statement, consistent with the team report.

Both students must participate in the **oral presentation**. Demonstrate installation/model connection, one tool-using run, and the architecture diagram. A recorded backup may be used if the live model or VM is unavailable. Follow the instructor’s Canvas announcement for the due date, presentation length, VM access details, and submission link.

8 Evaluation Rubric (100 Points)

Criterion	Evidence expected	Points
Installation and model connection	Correct agent installation, documented version/environment/model, working connection, and reproducible verification.	20
Function exploration	At least three capabilities explored, including one actual tool invocation; clear explanation of configuration and behavior.	15
Tool-use tests and evidence	Three distinct tasks, including tool use and cross-file interpretation; exact prompts, observed calls, results, and a documented error or limitation where feasible.	20
Tool-calling architecture	Accurate labeled diagram, component responsibilities, tool/result path, numbered real call sequence, and observation versus inference.	20
Technical report and repository	Required report structure, consistent citations, clear results, GitHub link/commit, reproducibility, safe evidence, and contribution/AI-use disclosure.	15
Presentation and participation	Both students present, demonstrate a tool call, explain the diagram, and document substantially equal contributions.	10
Total		100

A failed or refused tool call can earn credit when documented and analyzed. Invented evidence of successful tool use cannot.

9 Official Starting Points

1. Nous Research, “Hermes Agent Quickstart,” <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/getting-started/quickstart.md>.
2. Nous Research, “Hermes Agent: Hooks,” <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/features/hooks.md>.
3. OpenClaw, “Onboarding (CLI),” <https://docs.openclaw.ai/start/wizard>.

4. OpenClaw, “Tools overview,” <https://docs.openclaw.ai/tools>.
5. OpenClaw, “Local models,” <https://docs.openclaw.ai/gateway/local-models>.
6. OpenClaw, “Trajectory bundles,” <https://docs.openclaw.ai/tools/trajectory>.

Use the instructions that match the version you install; agent commands and model support may change.